

## Task 1 – Configure Classic Load Balancer

### Objective

To configure an Internet-facing Classic Load Balancer (CLB) in AWS and distribute incoming HTTP traffic between two Apache web servers running in different Availability Zones.

### Environment

- AWS Region: N. Virginia (us-east-1)
- VPC: CLB-VPC
- VPC CIDR: 10.10.0.0/16
- Availability Zones: us-east-1a and us-east-1b

### EC2 Servers

| Server       | Availability Zone | Web Server |
|--------------|-------------------|------------|
| CLB-Server-1 | us-east-1a        | Apache     |
| CLB-Server-2 | us-east-1b        | Apache     |

### Implementation Steps

#### Step 1 – Create the VPC

1. Open the AWS Management Console.
2. Navigate to VPC → Your VPCs.
3. Click Create VPC.
4. Select VPC only.
5. Configure:
  - a. Name: CLB-VPC
  - b. IPv4 CIDR: 10.10.0.0/16
  - c. IPv6: No IPv6 CIDR block
  - d. Tenancy: Default
6. Click Create VPC.

## Step 2 – Create Public Subnets

Created two public subnets in different Availability Zones.

### *Public Subnet 1*

- Name: Public Subnet 1
- Availability Zone: us-east-1a
- CIDR: 10.10.1.0/24

### *Public Subnet 2*

- Name: Public Subnet 2
- Availability Zone: us-east-1b
- CIDR: 10.10.2.0/24

Enabled Auto-assign public IPv4 address for the public subnet so that EC2 instances could communicate with the internet.

## Step 3 – Create and Attach Internet Gateway

1. Navigate to VPC → Internet Gateways.
2. Created an Internet Gateway named:
  - a. CLB-IGW
3. Attached CLB-IGW to CLB-VPC.

## Step 4 – Configure Public Route Table

Created a route table:

- Name: CLB-Public-RT
- VPC: CLB-VPC

Added the following route:

| Destination  | Target |
|--------------|--------|
| 10.10.0.0/16 | Local  |

0.0.0.0/0          CLB-  
IGW

Associated both public subnets with CLB-Public-RT.

### Step 5 – Configure Security Group

Created security group:

CLB-Web-SG

Inbound rules:

| Type | Port | Source    |
|------|------|-----------|
| HTTP | 80   | 0.0.0.0/0 |
| SSH  | 22   | My IP     |

HTTP port 80 was allowed so that users could access the Apache web servers through the load balancer.

### Step 6 – Configure Network ACL

The public subnet was associated with a Network ACL allowing required traffic.

Inbound:

- Rule 100
- All traffic
- Source: 0.0.0.0/0
- Allow

Outbound:

- Rule 100
- All traffic
- Destination: 0.0.0.0/0
- Allow

## Step 7 – Launch EC2 Server 1

Created EC2 instance:

CLB-Server-1

- AMI: Amazon Linux 2023
- Instance type: t3.micro
- Availability Zone: us-east-1a
- Subnet: Public Subnet 1
- Security Group: CLB-Web-SG

Apache was installed using EC2 User Data:

```
#!/bin/bash
```

```
dnf install -y httpd
```

```
systemctl enable httpd
```

```
systemctl start httpd
```

```
echo "<h1>Hello from CLB Server 1</h1>" > /var/www/html/index.html
```

## Step 8 – Launch EC2 Server 2

Created EC2 instance:

CLB-Server-2

- AMI: Amazon Linux 2023
- Instance type: t3.micro
- Availability Zone: us-east-1b
- Subnet: Public Subnet 2
- Security Group: CLB-Web-SG

Apache was installed using:

```
#!/bin/bash
```

```
dnf install -y httpd
```

```
systemctl enable httpd
systemctl start httpd
```

```
echo "<h1>Hello from CLB Server 2</h1>" > /var/www/html/index.html
```

Both EC2 instances were verified as running and Apache was accessible.

## Step 9 – Create Classic Load Balancer

Navigated to:

EC2 → Load Balancers → Create Load Balancer → Classic Load Balancer

Configured:

- Name: CLB-Classic-LB
- Scheme: Internet-facing
- VPC: CLB-VPC
- Availability Zones:
  - us-east-1a → Public Subnet 1
  - us-east-1b → Public Subnet 2
- Security Group: CLB-Web-SG

## Step 10 – Configure Listener

Configured the Classic Load Balancer listener:

| Load Balancer Protocol | Load Balancer Port | Instance Protocol | Instance Port |
|------------------------|--------------------|-------------------|---------------|
| HTTP                   | 80                 | HTTP              | 80            |

This allows the load balancer to receive HTTP requests on port 80 and forward them to the Apache servers on port 80.

## Step 11 – Configure Health Check

Configured the health check as:

- Ping Protocol: HTTP
- Ping Port: 80
- Ping Path: /
- Timeout: 2 seconds
- Interval: 5 seconds
- Unhealthy threshold: 2
- Healthy threshold: 10

The health check verifies whether the Apache web servers are available to receive traffic.

## Step 12 – Register EC2 Instances

Registered both EC2 instances with the Classic Load Balancer:

- CLB-Server-1
- CLB-Server-2

Both instances were registered on port 80.

The instances were placed in different Availability Zones to provide availability across zones.

## Step 13 – Configure Load Balancer Attributes

Configured:

- Cross-zone load balancing: Enabled
- Connection draining: Enabled
- Connection draining timeout: 300 seconds

Cross-zone load balancing allows the load balancer to distribute requests across registered instances in the enabled Availability Zones.

## Step 14 – Create and Verify Load Balancer

Created the Classic Load Balancer successfully.

The load balancer showed:

Status: 2 of 2 instances in service

This confirmed that both registered EC2 instances passed the configured health checks.

## Step 15 – Test Classic Load Balancer

Classic Load Balancer DNS name:

CLB-Classic-LB-781490472.us-east-1.elb.amazonaws.com

Opened the DNS name using HTTP:

<http://CLB-Classic-LB-781490472.us-east-1.elb.amazonaws.com>

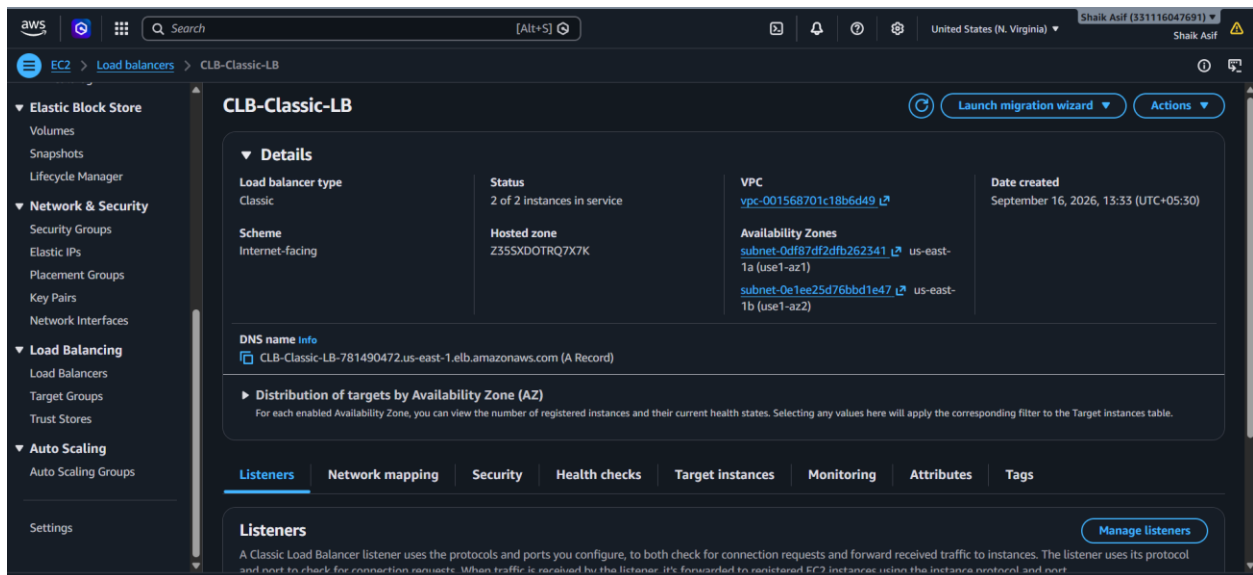
The request successfully reached the Apache web servers through the Classic Load Balancer.

The response displayed the server-specific Apache page, such as:

Hello from CLB Server 1

or

Hello from CLB Server 2







## Result

The Classic Load Balancer was successfully configured as an Internet-facing load balancer in the custom CLB-VPC.

Two Apache web servers running in separate Availability Zones were registered with the load balancer. Both instances passed the health checks and were shown as InService.

The Classic Load Balancer DNS name was successfully tested, confirming that HTTP traffic was being forwarded from the load balancer to the backend Apache instances.

## Conclusion

The Classic Load Balancer configuration was successfully completed. The setup demonstrated how a Classic Load Balancer receives HTTP traffic from users and distributes the traffic to

healthy backend EC2 instances across multiple Availability Zones, providing load distribution and improved availability.

## **Task 2 – Configure Application Load Balancer (ALB)**

### Objective

To configure an Application Load Balancer (ALB) in AWS to distribute HTTP traffic across two Apache web servers running in different Availability Zones.

### Environment

- AWS Region: N. Virginia (us-east-1)
- VPC: CLB-VPC
- VPC CIDR: 10.10.0.0/16
- Load Balancer: ALB-Application-LB
- Load Balancer Type: Application
- Scheme: Internet-facing
- IP Address Type: IPv4

### Steps / Implementation

#### 1. Created Application Load Balancer

1. Opened AWS Console → EC2 → Load Balancers.
2. Selected Create Load Balancer.
3. Selected Application Load Balancer.
4. Configured:
  - a. Name: ALB-Application-LB
  - b. Scheme: Internet-facing
  - c. IP address type: IPv4
5. Selected VPC:
  - a. CLB-VPC
6. Selected two Availability Zones:
  - a. us-east-1a → Public Subnet 1
  - b. us-east-1b → Public Subnet 2
7. Selected security group:

### a. CLB-Web-SG

## 2. Created Target Group

Created a target group with the following configuration:

- Target group name: ALB-Target-Group
- Target type: Instances
- Protocol: HTTP
- Port: 80
- VPC: CLB-VPC
- Health check protocol: HTTP
- Health check path: /

## 3. Registered EC2 Instances

Registered the following EC2 instances with the target group:

- CLB-Server-1
- CLB-Server-2

Both instances were registered on HTTP port 80.

## 4. Configured ALB Listener

Configured the ALB listener:

- Protocol: HTTP
- Port: 80
- Default action: Forward traffic to ALB-Target-Group

Therefore, when a user sends an HTTP request to the ALB, the ALB forwards the request to one of the healthy EC2 instances.

## 5. Verified Target Health

Opened:

EC2 → Target Groups → ALB-Target-Group → Targets

Verified that:

- CLB-Server-1 → Healthy
- CLB-Server-2 → Healthy

Both targets successfully passed the configured HTTP health check.

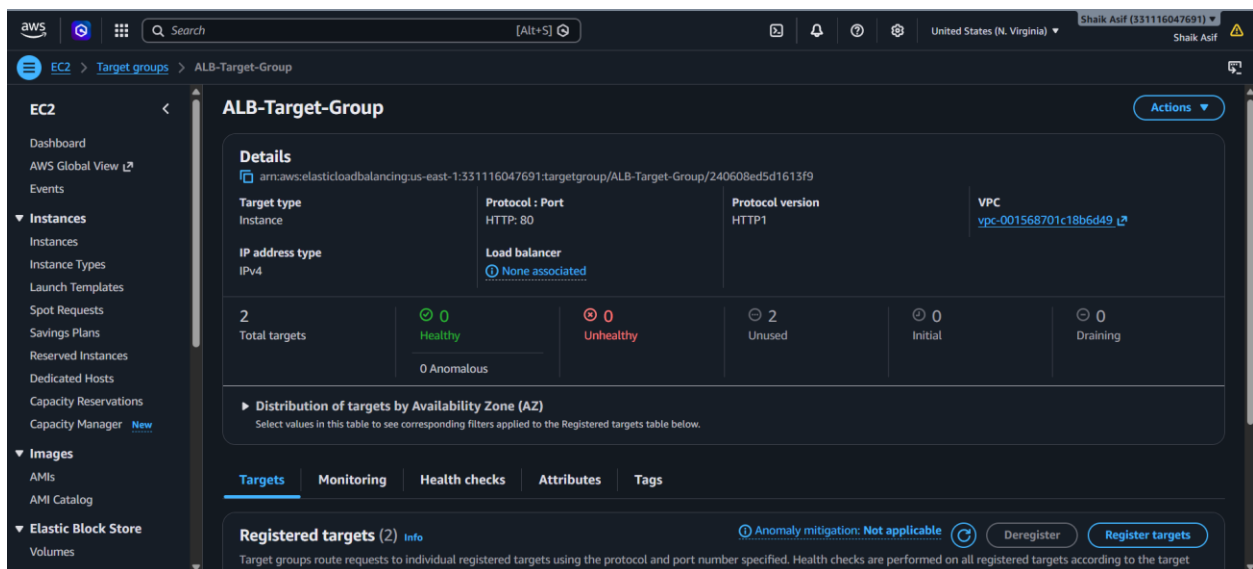
## 6. Tested the Application Load Balancer

Used the ALB DNS name:

ALB-Application-LB-94532066.us-east-1.elb.amazonaws.com

Opened the DNS name in a web browser and verified that the Apache web application was accessible through the ALB.

The application displayed the server response from the registered EC2 instances.



aws

Search

[Alt+S]

United States (N. Virginia)

Shaik Asif (331116047691)

Shaik Asif

EC2 > Load balancers > Create Application Load Balancer

## Create Application Load Balancer Info

The Application Load Balancer distributes incoming HTTP and HTTPS traffic across multiple targets such as Amazon EC2 instances, microservices, and containers, based on request attributes. When the load balancer receives a connection request, it evaluates the listener rules in priority order to determine which rule to apply, and if applicable, it selects a target from the target group for the rule action.

▶ How Application Load Balancers work

### Basic configuration

**Load balancer name**  
Name must be unique within your AWS account and can't be changed after the load balancer is created.

ALB-Application-LB

A maximum of 32 alphanumeric characters including hyphens are allowed, but the name must not begin or end with a hyphen.

**Scheme** Info  
Scheme can't be changed after the load balancer is created.

☒ **Internet-facing**

- Serves internet-facing traffic.
- Has public IP addresses.
- DNS name resolves to public IPs.
- Requires a public subnet.

☐ **Internal**

- Serves internal traffic.
- Has private IP addresses.
- DNS name resolves to private IPs.
- Compatible with the IPv4 and Dualstack IP address types.

**Load balancer IP address type** Info  
Select the front-end IP address type to assign to the load balancer. The VPC and subnets mapped to this load balancer must include the selected IP address types. Public IPv4 addresses have an additional cost.

☒ **IPv4**  
Includes only IPv4 addresses.

aws

Search

[Alt+S]

United States (N. Virginia)

Shaik Asif (331116047691)

Shaik Asif

EC2 > Load balancers > ALB-Application-LB

▼ Elastic Block Store

Volumes

Snapshots

Lifecycle Manager

▼ Network & Security

Security Groups

Elastic IPs

Placement Groups

Key Pairs

Network Interfaces

▼ Load Balancing

Load Balancers

Target Groups

Trust Stores

▼ Auto Scaling

Auto Scaling Groups

Settings

## ALB-Application-LB Actions

▼ Details

**Load balancer type**  
Application

**Scheme**  
Internet-facing

**Status**  
Active

**Hosted zone**  
Z355XDOTRQ7X7K

**VPC**  
[vpc-001568701c18b6d49](#)

**Availability Zones**  
[subnet-0df87df2dfb262341](#) us-east-1a (use1-az1)  
[subnet-0e1ee25d76bbd1e47](#) us-east-1b (use1-az2)

**Load balancer IP address type**  
IPv4

**Date created**  
September 16, 2026, 14:28 (UTC+05:30)

**Load balancer ARN**  
[arn:aws:elasticloadbalancing:us-east-1:331116047691:loadbalancer/app/ALB-Application-LB/452102c434d4093a](#)

**DNS name** Info  
[ALB-Application-LB-94532066.us-east-1.elb.amazonaws.com](#) (A Record)

Listeners and rules

Network mapping

Resource map

Security

Monitoring

Integrations

Attributes

Capacity

Tags

**Listeners and rules (1)** Info

Manage rules

Manage listener

Add listener

A listener checks for connection requests on its configured protocol and port. Incoming requests are evaluated against rules in priority order; the first matching rule determines routing. If no rule matches, the default action is applied.

EC2

Target groups

ALB-Target-Group

▼ Elastic Block Store

Volumes

Snapshots

Lifecycle Manager

▼ Network & Security

Security Groups

Elastic IPs

Placement Groups

Key Pairs

Network Interfaces

▼ Load Balancing

Load Balancers

Target Groups

Trust Stores

▼ Auto Scaling

Auto Scaling Groups

Settings

ALB-Target-Group

Actions

Details

arn:aws:elasticloadbalancing:us-east-1:331116047691:targetgroup/ALB-Target-Group/240608ed5d1613f9

Target type

Instance

Protocol : Port

HTTP: 80

Protocol version

HTTP1

VPC

vpc-001568701c18b6d49

IP address type

IPv4

Load balancer

ALB-Application-LB

2

Total targets

2

Healthy

0 Anomalous

0

Unhealthy

0

Unused

0

Initial

0

Draining

Distribution of targets by Availability Zone (AZ)

Select values in this table to see corresponding filters applied to the Registered targets table below.

Targets

Monitoring

Health checks

Attributes

Tags

Registered targets (2)

Info

Anomaly mitigation: Not applicable

Deregister

Register targets

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target

← → ↺ Not secure alb-application-lb-94532066.us-east-1.elb.amazonaws.com ☆ 📄 🖨️ 🌐

Hello from CLB Server 1 Made by Asifshaik

← → ↺ Not secure alb-application-lb-94532066.us-east-1.elb.amazonaws.com

Hello from CLB Server 2 Made by Asifshaik

## Result

The Application Load Balancer was successfully configured in the custom CLB-VPC.

The ALB is internet-facing and distributes HTTP traffic on port 80 to the two healthy Apache EC2 instances located in different Availability Zones.

## Conclusion

The Application Load Balancer was successfully implemented and tested. Both EC2 instances were registered with the target group and reported Healthy, and the application was successfully accessed through the ALB DNS name.

## **Task 3 – Configure Network Load Balancer (NLB)**

### Objective

To configure a Network Load Balancer (NLB) to distribute TCP traffic across two Apache web servers running in different Availability Zones.

### Environment

- AWS Region: N. Virginia (us-east-1)
- VPC: CLB-VPC
- VPC CIDR: 10.10.0.0/16
- NLB Name: NLB-Network-LB
- Scheme: Internet-facing
- IP Address Type: IPv4

### Steps / Implementation

#### 1. Created NLB Target Group

Created a target group with the following configuration:

- Target Group Name: NLB-Target-Group
- Target Type: Instances
- Protocol: TCP
- Port: 80
- VPC: CLB-VPC
- Health Check Protocol: TCP

## 2. Registered EC2 Instances

Registered the following EC2 instances:

- CLB-Server-1
- CLB-Server-2

Both instances were registered on TCP port 80.

## 3. Created Network Load Balancer

Created an internet-facing Network Load Balancer:

- Name: NLB-Network-LB
- Scheme: Internet-facing
- IP Address Type: IPv4
- VPC: CLB-VPC

Selected two Availability Zones:

- us-east-1a → Public Subnet 1
- us-east-1b → Public Subnet 2

## 4. Configured Listener

Configured the NLB listener:

- Protocol: TCP
- Port: 80
- Default Action: Forward traffic to NLB-Target-Group

## 5. Verified Target Health

Opened:



EC2 → Target Groups → NLB-Target-Group → Targets

Verified that both registered instances were Healthy:

- CLB-Server-1 → Healthy
- CLB-Server-2 → Healthy

This confirmed that the NLB could successfully communicate with the registered targets.

## 6. Tested NLB

Copied the DNS name of NLB-Network-LB and accessed it using HTTP in a web browser.

The Apache application was successfully displayed through the NLB, confirming that traffic was reaching the backend server.

The screenshot displays the AWS Management Console interface for an NLB-Target-Group. The left sidebar shows the navigation menu with categories like Elastic Block Store, Network & Security, Load Balancing, and Auto Scaling. The main content area is titled 'NLB-Target-Group' and includes an 'Actions' button. Below the title, the 'Details' section shows the target type as 'Instance', protocol as 'TCP: 80', VPC as 'vpc-001568701c18b6d49', and IP address type as 'IPv4'. A status summary table indicates 2 total targets, all of which are healthy. Below this, there is a section for 'Distribution of targets by Availability Zone (AZ)' with a note to select values to see corresponding filters. At the bottom, the 'Targets' tab is active, showing 'Registered targets (2)' with a search bar and buttons for 'Deregister' and 'Register targets'.

| Total targets | Healthy | Unhealthy | Unused | Initial | Draining |
|---------------|---------|-----------|--------|---------|----------|
| 2             | 2       | 0         | 2      | 0       | 0        |

aws [Search] [Alt+S] United States (N. Virginia) Shaik Asif (331116047691) Shaik Asif

EC2 > Load balancers > NLB-Network-LB

### NLB-Network-LB

**Details**

|                                      |                                      |                                                                                                                               |                                                              |
|--------------------------------------|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| <b>Load balancer type</b><br>Network | <b>Status</b><br>Active              | <b>VPC</b><br>vpc-001568701c18b6d49                                                                                           | <b>Load balancer IP address type</b><br>IPv4                 |
| <b>Scheme</b><br>Internet-facing     | <b>Hosted zone</b><br>Z26RNL4JYFTOTI | <b>Availability Zones</b><br>subnet-0df87df2dfb262341 us-east-1a (use1-az1)<br>subnet-0e1ee25d76bbd1e47 us-east-1b (use1-az2) | <b>Date created</b><br>September 16, 2026, 14:42 (UTC+05:30) |

**Load balancer ARN**  
arn:aws:elasticloadbalancing:us-east-1:331116047691:loadbalancer/net/NLB-Network-LB/c16cd858a8d87acf

**DNS name**  
NLB-Network-LB-c16cd858a8d87acf.elb.us-east-1.amazonaws.com (A Record)

**Listeners and rules (1)**

A listener checks for connection requests using the protocol and port that you configure. Traffic received by a Network Load Balancer listener is forwarded to the selected target group.

Filter listeners

aws [Search] [Alt+S] United States (N. Virginia) Shaik Asif (331116047691) Shaik Asif

EC2 > Target groups > NLB-Target-Group

### NLB-Target-Group

**Details**

arn:aws:elasticloadbalancing:us-east-1:331116047691:targetgroup/NLB-Target-Group/d9b4ab9d82c9fe6b

|                                |                                   |                                     |                                |
|--------------------------------|-----------------------------------|-------------------------------------|--------------------------------|
| <b>Target type</b><br>Instance | <b>Protocol : Port</b><br>TCP: 80 | <b>VPC</b><br>vpc-001568701c18b6d49 | <b>IP address type</b><br>IPv4 |
|--------------------------------|-----------------------------------|-------------------------------------|--------------------------------|

**Load balancer**  
NLB-Network-LB

|                           |                     |                       |                    |                     |                      |
|---------------------------|---------------------|-----------------------|--------------------|---------------------|----------------------|
| <b>Total targets</b><br>2 | <b>Healthy</b><br>2 | <b>Unhealthy</b><br>0 | <b>Unused</b><br>0 | <b>Initial</b><br>0 | <b>Draining</b><br>0 |
|---------------------------|---------------------|-----------------------|--------------------|---------------------|----------------------|

**Distribution of targets by Availability Zone (AZ)**  
Select values in this table to see corresponding filters applied to the Registered targets table below.

**Targets** Monitoring Health checks Attributes Tags

**Registered targets (2)**

Filter targets

← → ↺ Not secure nlb-network-lb-c16cd858a8d87acf.elb.us-east-1.amazonaws.com ☆ 📄 📄 📄 📄

Hello from CLB Server 1 Made by Asifshaik

← → ↻ ⚠ Not secure nlb-network-lb-c16cd858a8d87acf.elb.us-east-1.amazonaws.com

## Hello from CLB Server 2 Made by Asifshaik

### Result

The Network Load Balancer was successfully configured and tested. Both EC2 instances were registered as targets and passed the TCP health checks.

The application was successfully accessed using the NLB DNS name.

### Conclusion

The Network Load Balancer was successfully implemented using TCP port 80 and configured across two Availability Zones. It successfully forwarded client traffic to the healthy Apache EC2 instances.

One-line explanation:

NLB receives network-level TCP traffic and forwards it to healthy backend targets with high performance and low latency.

## Task 4 - Attach SSL Certificate to Application Load Balancer

### Objective

To configure an HTTPS listener on the Application Load Balancer (ALB) using an AWS Certificate Manager (ACM) SSL/TLS certificate and secure client-to-ALB communication over HTTPS.

### Environment

- AWS Region: N. Virginia (us-east-1)
- VPC: CLB-VPC
- Application Load Balancer: ALB-Application-LB
- Target Group: ALB-Target-Group
- Domain: asifshaik.online
- Certificate Provider: AWS Certificate Manager (ACM)

### Implementation Steps

#### Step 1: Verify SSL Certificate in ACM

1. Open the AWS Management Console.
2. Navigate to Certificate Manager (ACM).
3. Select the N. Virginia (us-east-1) region.
4. Verify that the certificate for asifshaik.online is available.
5. Confirm that the certificate status is Issued.

Certificate details:

- Domain: asifshaik.online
- Status: Issued
- Type: Amazon Issued
- Key Algorithm: RSA 2048
- Certificate ID: e0b25058-9c59-480c-9591-1d87df7b5f78

Evidence: Screenshot of ACM showing the certificate status as Issued.

#### Step 2: Open the Application Load Balancer

1. Navigate to EC2 → Load Balancers.
2. Select ALB-Application-LB.

3. Open the Listeners and rules section.
4. The existing HTTP listener was:
  - a. HTTP : 80
  - b. Forward traffic to ALB-Target-Group

### Step 3: Add HTTPS Listener

1. Click Add listener.
2. Select protocol HTTPS.
3. Set the port to 443.
4. Configure the default action:
  - a. Action: Forward
  - b. Target group: ALB-Target-Group
5. Select the ACM certificate for:
  - a. asifshaik.online
6. Keep the SSL/TLS security policy as:
  - a. ELBSecurityPolicy-TLS13-1-2-Res-PQ-2025-09
7. Keep mutual TLS (mTLS) disabled.
8. Create the listener.

Evidence: Screenshot showing:

- HTTPS : 443
- Forward to ALB-Target-Group
- SSL certificate: asifshaik.online

### Step 4: Allow HTTPS in Security Group

The ALB security group was updated to allow HTTPS traffic.

Security Group:

CLB-Web-SG

Inbound rule added:

| Type  | Protocol | Port | Source    |
|-------|----------|------|-----------|
| HTTPS | TCP      | 443  | 0.0.0.0/0 |

The existing HTTP port 80 rule was retained.

Evidence: Screenshot of the CLB-Web-SG inbound rules showing TCP port 443 allowed.

## Step 5: Verify ALB Listeners

The Application Load Balancer now contains two listeners:

| Listener | Port | Action                      |
|----------|------|-----------------------------|
| HTTP     | 80   | Forward to ALB-Target-Group |
| HTTPS    | 443  | Forward to ALB-Target-Group |

The HTTPS listener uses the ACM certificate for asifshaik.online.

## Step 6: Test HTTPS

The HTTPS endpoint was tested using the ALB DNS name:

<https://ALB-Application-LB-94532066.us-east-1.elb.amazonaws.com>

The browser displayed:

ERR\_CERT\_COMMON\_NAME\_INVALID

This does not indicate that the ACM certificate configuration failed.

The certificate is issued for:

asifshaik.online

while the test was performed using the AWS-generated ALB hostname:

ALB-Application-LB-94532066.us-east-1.elb.amazonaws.com

Therefore, the certificate hostname does not match the ALB's AWS-generated DNS name.

The certificate will match correctly after the domain asifshaik.online is mapped to the Application Load Balancer through Route 53 in the next task.

## Traffic Flow

User / Browser

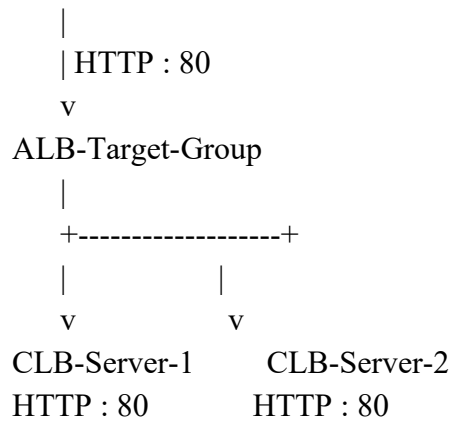
|

| HTTPS : 443

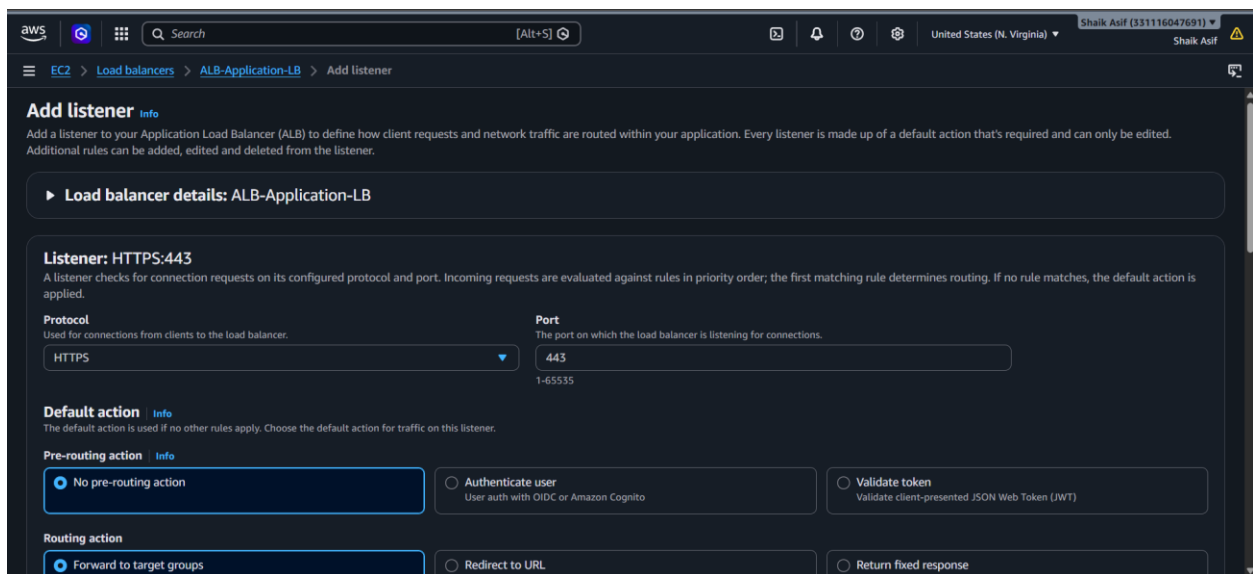
v

Application Load Balancer

(ALB-Application-LB)



SSL/TLS termination occurs at the Application Load Balancer. The ALB then forwards the request to the backend EC2 instances over HTTP port 80.



aws

Search

[Alt+S]

United States (N. Virginia)

Shaik Asif (331116047691)

Shaik Asif

EC2

Load balancers

ALB-Application-LB

Add listener

Secure listener settings

Info

Security policy

Info

Your load balancer uses a Secure Socket Layer (SSL) negotiation configuration called a security policy to manage SSL connections with clients. [Compare security policies](#)

Security category

Policy name

New

All security policies

ELBSecurityPolicy-TLS13-1-2-Res-PQ-2025-09 (Recommended)

Default SSL/TLS server certificate

The certificate used if a client connects without SNI protocol, or if there are no matching certificates. You can source this certificate from AWS Certificate Manager (ACM), Amazon Identity and Access Management (IAM), or import a certificate. This certificate will automatically be added to your listener certificate list.

Certificate source

From ACM

From IAM

Import certificate

Certificate (from ACM)

The selected certificate will be applied as the default SSL/TLS server certificate for this load balancer's secure listeners.

asifshaik.online  
e0b25058-9c59-480c-9591-1d87df7b5f78

Request new ACM certificate

Client certificate handling

Info

Client certificates are used to make authenticated requests to remote servers. [Learn more](#)

☐ Mutual authentication (mTLS)

Mutual TLS (Transport Layer Security) authentication offers two-way peer authentication. It adds a layer of security over TLS and allows your services to verify the client that's making the connection.

aws

Search

[Alt+S]

United States (N. Virginia)

Shaik Asif (331116047691)

Shaik Asif

EC2

Security Groups

sg-09ead70d63c921044

Edit inbound rules

Edit inbound rules

Info

Inbound rules control the incoming traffic that's allowed to reach the instance.

Inbound rules

Info

| Security group rule ID | Type  | Protocol | Port range | Source | Description - optional |                   |        |
|------------------------|-------|----------|------------|--------|------------------------|-------------------|--------|
| sg-02f7416b47e69a31c   | SSH   | TCP      | 22         | Custom | Q                      | 103.160.27.240/32 | Delete |
| sg-0703fbc8bf314cd98   | HTTPS | TCP      | 443        | Custom | Q                      | 0.0.0.0/0         | Delete |
| sg-094c55ac15b89b1e3   | HTTP  | TCP      | 80         | Custom | Q                      | 0.0.0.0/0         | Delete |

Add rule

Cancel Preview changes Save rules

aws

Search

[Alt+S]

United States (N. Virginia)

Shaik Asif (331116047691)

Shaik Asif

EC2

Load balancers

ALB-Application-LB

Capacity Reservations

Capacity Manager

Images

AMIs

AMI Catalog

Elastic Block Store

Volumes

Snapshots

Lifecycle Manager

Network & Security

Security Groups

Elastic IPs

Placement Groups

Key Pairs

Network Interfaces

Load Balancing

Load Balancers

Target Groups

Trust Stores

Auto Scaling

ALB-Application-LB

Actions

Details

Load balancer type

Application

Status

Active

VPC

vpc-001568701c18b6d49

Scheme

Internet-facing

Hosted zone

Z35SXDOTRQ7X7K

Availability Zones

subnet-0df87df2dfb262341 us-east-1a (use1-az1)

subnet-0e1ee25d76bbd1e47 us-east-1b (use1-az2)

Load balancer IP address type

IPv4

Date created

September 16, 2026, 14:28 (UTC+05:30)

Load balancer ARN

arn:aws:elasticloadbalancing:us-east-1:331116047691:loadbalancer/app/ALB-Application-LB/452102c434d4093a

DNS name

ALB-Application-LB-94532066.us-east-1.elb.amazonaws.com (A Record)

Listeners and rules

Network mapping

Resource map

Security

Monitoring

Integrations

Attributes

Capacity

Tags

Listeners and rules (2)

Info

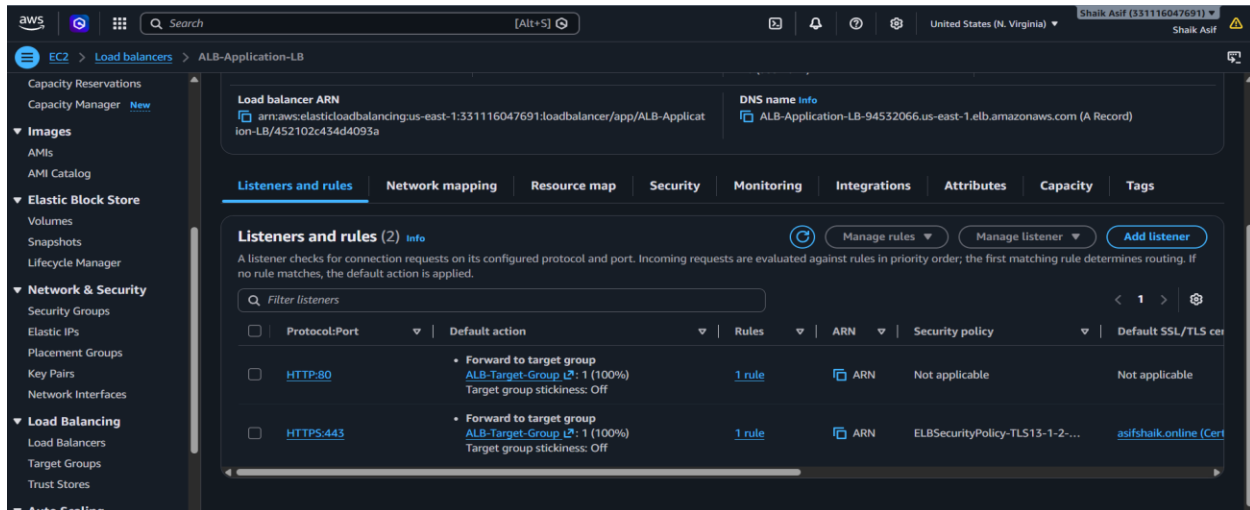
Manage rules

Manage listener

Add listener

A listener checks for connection requests on its configured protocol and port. Incoming requests are evaluated against rules in priority order; the first matching rule determines routing. If no rule matches, the default action is applied.





## Result

The SSL/TLS certificate for asifshaik.online was successfully attached to the Application Load Balancer. An HTTPS listener on port 443 was configured and traffic is forwarded to ALB-Target-Group.

## Conclusion

SSL/TLS was successfully configured for the Application Load Balancer using AWS Certificate Manager. The ALB can now accept secure HTTPS traffic on port 443 and forward the requests to the backend EC2 instances. The certificate hostname mismatch observed while testing the AWS ALB DNS name is expected and will be resolved when asifshaik.online is mapped to the ALB using Route 53.

## Task 5 - Map Application Load Balancer to Route 53

### Objective

To map the Application Load Balancer (ALB) to the custom domain asifshaik.online using Amazon Route 53 and enable access to the application through the domain name.

### Environment

- AWS Region: N. Virginia (us-east-1)
- VPC: CLB-VPC
- Application Load Balancer: ALB-Application-LB
- Domain: asifshaik.online

- DNS Provider/Registrar: GoDaddy
- DNS Service: Amazon Route 53
- SSL Certificate: ACM certificate for asifshaik.online

## Implementation Steps

### Step 1: Create Route 53 Hosted Zone

1. Opened AWS Console → Route 53.
2. Selected Hosted zones.
3. Created a new hosted zone:
  - a. Domain name: asifshaik.online
  - b. Type: Public hosted zone
4. Route 53 automatically created the required NS and SOA records.

### Step 2: Obtain Route 53 Nameservers

The hosted zone provided the following nameservers:

ns-381.awsdns-47.com  
ns-1210.awsdns-23.org  
ns-1992.awsdns-57.co.uk  
ns-954.awsdns-55.net

### Step 3: Update GoDaddy Nameservers

1. Opened the GoDaddy domain management page.
2. Selected asifshaik.online.
3. Opened Nameservers.
4. Selected Custom nameservers.
5. Updated the nameservers with the four Route 53 nameservers listed above.
6. Saved the changes.

### Step 4: Create Route 53 A Record

Created an A record in the Route 53 hosted zone.

Configuration:

- Record name: Blank (apex/root domain)
- Record type: A

- Alias: Enabled
- Route traffic to: Application and Classic Load Balancer
- Region: N. Virginia (us-east-1)
- Load Balancer: ALB-Application-LB
- Evaluate target health: No

The A record maps:

asifshaik.online



ALB-Application-LB



ALB-Target-Group



CLB-Server-1 / CLB-Server-2

### Step 5: Verify DNS Delegation

Used Git Bash to verify that the domain was delegated to the Route 53 nameservers.

Command:

```
nslookup -type=NS asifshaik.online 8.8.8.8
```

The response returned:

```
asifshaik.online      nameserver = ns-381.awsdns-47.com
asifshaik.online      nameserver = ns-1992.awsdns-57.co.uk
asifshaik.online      nameserver = ns-1210.awsdns-23.org
asifshaik.online      nameserver = ns-954.awsdns-55.net
```

This confirmed that Google DNS was able to resolve the Route 53 nameservers for the domain.

### Step 6: Verify Domain Resolution

Used:

```
nslookup asifshaik.online 8.8.8.8
```

The domain successfully resolved to the ALB.

Finally, accessed:

<https://asifshaik.online>

The application was successfully reached through the custom domain.

### DNS Flow

User Browser

|

| <https://asifshaik.online>

↓

GoDaddy Domain

|

| Nameserver delegation

↓

Amazon Route 53

|

| A Alias Record

↓

Application Load Balancer  
(ALB-Application-LB)

|

↓

ALB-Target-Group

|

| — CLB-Server-1

|

| — CLB-Server-2

Route 53

Hosted zones

asifshaik.online

Public

asifshaik.online

Info

Delete zone

Test record

Configure query logging

Hosted zone details

Edit hosted zone

Records (2)

Accelerated recovery

DNSSEC signing

Hosted zone tags (0)

Records (2)

Info

Delete record

Import zone file

Create record

Automatic mode is the current search behavior optimized for best filter results. To change modes go to settings.

Filter records by property or value

Type

Routing p...

Alias

< 1 >

☐

Record ...

Type

Routing...

Differ...

Alias

Value/Route traffic to

☐

asifshaik...

NS

Simple

-

No

ns-381.awsdns-47.com.  
ns-1210.awsdns-23.org.  
ns-1992.awsdns-57.co.uk.  
ns-954.awsdns-55.net.

☐

asifshaik...

SOA

Simple

-

No

ns-381.awsdns-47.com. aw

Route 53

Dashboard

Hosted zones

Health checks

Profiles

Global Resolver

Global resolvers

Shared DNS views

VPC Resolver

VPCs

Inbound endpoints

Outbound endpoints

Rules

Query logging

Outposts

Domains

Registered domains

Requests

IP-based routing

Route 53

Hosted zones

asifshaik.online

Create record

Create record

Info

Quick create record

Switch to wizard

Record 1

Delete

Record name

Info

subdomain

asifshaik.online

Record type

Info

A - Routes traffic to an IPv4 address and some AWS resources

Alias

Keep blank to create a record for the root domain.

Route traffic to

Info

Alias to Application and Classic Load Balancer

US East (N. Virginia)

dualstack-ALB-Application-LB-94532066.us-east-1.elb.amazonaws.com

Alias hosted zone ID: Z35SXDOTRQ7X7K

Routing policy

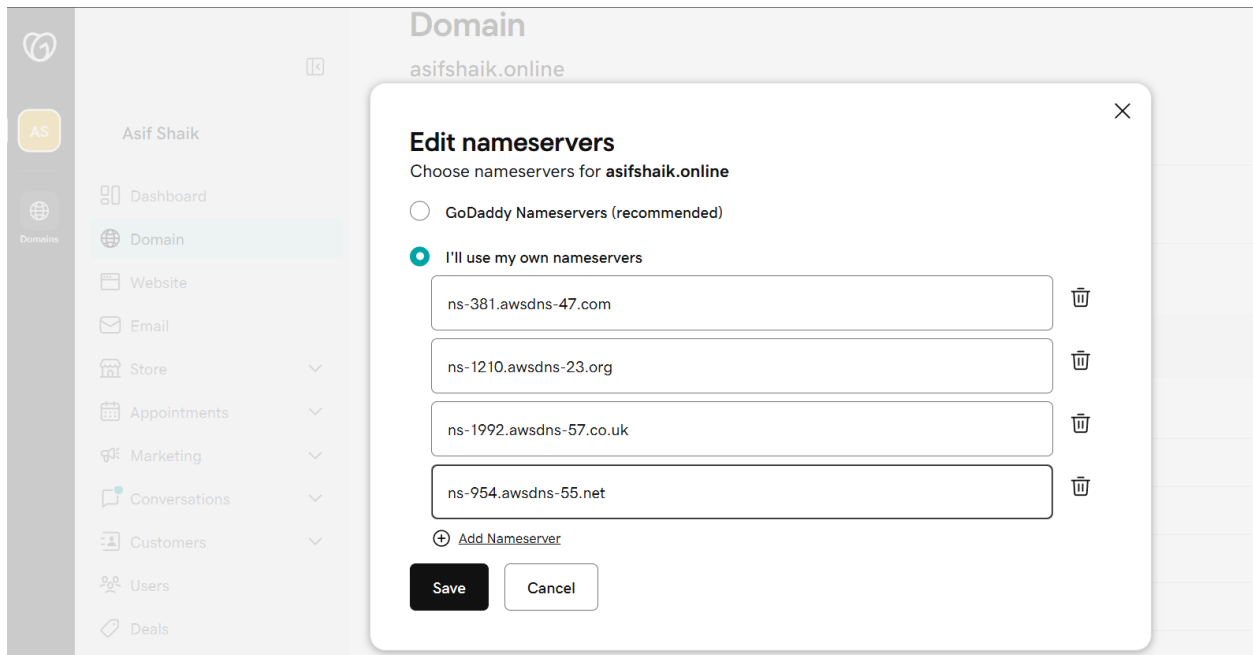
Info

Simple routing

Evaluate target health

Yes

Add another record



```
asifs@Asif_Shaik MINGW64 ~
$ nslookup -type=NS asifshaik.online 8.8.8.8
Non-authoritative answer:
Server: dns.google
Address: 8.8.8.8

asifshaik.online      nameserver = ns-381.awsdns-47.com
asifshaik.online      nameserver = ns-1992.awsdns-57.co.uk
asifshaik.online      nameserver = ns-1210.awsdns-23.org
asifshaik.online      nameserver = ns-954.awsdns-55.net

asifs@Asif_Shaik MINGW64 ~
$ nslookup asifshaik.online 8.8.8.8
Non-authoritative answer:
Server: dns.google
Address: 8.8.8.8

Name: asifshaik.online
Addresses: 52.21.39.239
           52.5.185.34
```



**Hello from CLB Server 1 Made by Asifshaik**



## Result

The Application Load Balancer was successfully mapped to the custom domain asifshaik.online using Amazon Route 53. DNS resolution was verified using Google DNS, and the application was successfully accessed through the custom domain.

## Conclusion

Route 53 was successfully configured to route requests for asifshaik.online to the Application Load Balancer. The GoDaddy domain was delegated to the Route 53 hosted zone, and an A Alias record was created to connect the domain with the ALB. The final domain-based HTTPS access was successfully verified.

## **Task 6 — Push Application Load Balancer Logs to S3**

### Objective

To configure the Application Load Balancer (ALB) to send access logs to an Amazon S3 bucket for storage and monitoring.

### Environment

- AWS Region: N. Virginia (us-east-1)

- Application Load Balancer: ALB-Application-LB
- S3 Bucket: alb-access-logs-asif-2026
- AWS Account ID: 331116047691

## Steps

### Step 1 – Create S3 Bucket

1. Open the AWS Management Console.
2. Go to S3.
3. Click Create bucket.
4. Enter bucket name:  
alb-access-logs-asif-2026
5. Select region:  
US East (N. Virginia) – us-east-1
6. Keep ACLs disabled.
7. Keep Block all public access enabled.
8. Keep default encryption as SSE-S3.
9. Click Create bucket.

### Step 2 – Configure S3 Bucket Policy

1. Open the bucket:  
alb-access-logs-asif-2026
2. Go to Permissions.
3. Open Bucket policy.
4. Added the following policy to allow the Elastic Load Balancing log delivery service to write objects:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "logdelivery.elasticloadbalancing.amazonaws.com"
      },
      "Action": "s3:PutObject",
      "Resource": "arn:aws:s3:::alb-access-logs-asif-2026/AWSLogs/331116047691/*"
    }
  ]
}
```



```
]
}
```

5. Click Save changes.

### Step 3 – Enable ALB Access Logs

1. Open EC2 → Load Balancers.
2. Select:  
ALB-Application-LB
3. Open the Attributes section.
4. Find Access logs.
5. Enable access logs.
6. Select destination type:  
S3
7. Select bucket:  
alb-access-logs-asif-2026
8. Leave the prefix blank.
9. Save the changes.

### Step 4 – Verify Log Delivery

After enabling access logs, AWS created the following test object in the S3 bucket:

AWSLogs/331116047691/ELBAccessLogTestFile

This test file confirmed that the ALB log delivery service could successfully access the S3 bucket and write an object.

### Step 5 – Generate ALB Traffic

1. Open the application using the Route 53 domain:

<https://asifshaik.online>

2. Refresh the page multiple times to generate ALB traffic.
3. Wait for the ALB access logs to be delivered to S3.
4. Open the S3 bucket and check the AWSLogs/331116047691/ path for the generated access-log files.

## Traffic Flow

User



Route 53



Application Load Balancer



Target Group



EC2 Server 1 / EC2 Server 2



ALB Access Logs



S3 Bucket



alb-access-logs-asif-2026

The screenshot shows the 'Create bucket' page in the AWS Management Console. The breadcrumb navigation at the top reads 'Amazon S3 > Buckets > Create bucket'. The page title is 'Create bucket' with an 'Info' link. Below the title, it states 'Buckets are containers for data stored in S3.' The 'General configuration' section is active and contains the following settings:

- AWS Region:** US East (N. Virginia) us-east-1
- Bucket type:** General purpose (selected). The 'Directory' option is also visible but not selected.
- Bucket namespace:** Global namespace (selected). The 'Account Regional namespace (recommended)' option is also visible but not selected.
- Bucket name:** alb-access-logs-asif-2026

Below the bucket name field, there is a note: 'Bucket names must be 3 to 63 characters and unique within the global namespace. Bucket names must also begin and end with a letter or number. Valid characters are a-z, 0-9, periods (.), and hyphens (-). Learn more'. At the bottom of the configuration section, there is a 'Copy settings from existing bucket - optional' section with a 'Choose bucket' button.

aws

Search

[Alt+S]

United States (N. Virginia)

Shaik Asif (33116047691)

Shaik Asif

Amazon S3

Buckets

alb-access-logs-asif-2026

Edit bucket policy

Edit bucket policy

Info

Bucket policy

The bucket policy, written in JSON, provides access to the objects stored in the bucket. Bucket policies don't apply to objects owned by other accounts. [Learn more](#)

Policy examples

Bucket ARN

arn:aws:s3::alb-access-logs-asif-2026

Policy

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Principal": {
7         "Service": "logdelivery.elasticloadbalancing.amazonaws.com"
8       },
9       "Action": "s3:PutObject",
10      "Resource": "arn:aws:s3::alb-access-logs-asif-2026/AWSLogs/33116047691/*"
11    }
12  ]
13 }
```

Edit statement

Select a statement

Select an existing statement in the policy or add a new statement.

+ Add new statement

aws

Search

[Alt+S]

United States (N. Virginia)

Shaik Asif (33116047691)

Shaik Asif

EC2

Load balancers

ALB-Application-LB

Elastic Block Store

Volumes

Snapshots

Lifecycle Manager

Network & Security

Security Groups

Elastic IPs

Placement Groups

Key Pairs

Network Interfaces

Load Balancing

Load Balancers

Target Groups

Trust Stores

Auto Scaling

Auto Scaling Groups

Preserve host header

Off

Availability Zone routing configuration

Cross-zone load balancing

On

ARC zonal shift integration

Disabled

Protection

Deletion protection

Off

Logging

Access logs - legacy

S3 location: alb-access-logs-asif-2026

Connection logs - legacy

Off

Health check logs - legacy

Off

aws

Search

[Alt+S]

United States (N. Virginia)

Shaik Asif (33116047691)

Shaik Asif

Amazon S3

...

AWSLogs/

33116047691/

elasticloadbalancing/

us-east-1/

2026/

09/

16/

33116047691\_elasticloadbalancing...

Amazon S3

Buckets

General purpose buckets

Directory buckets

Table buckets

Vector buckets

Files

File systems

Access management and security

Access Points

Access Points for FSx

Access Grants

IAM Access Analyzer

Storage management and insights

Storage Lens

Batch Operations

Account and permission

CloudShell

Agent Toolkit for AWS

Feedback

Console Mobile App

33116047691\_elasticloadbalancing\_us-east-1\_app.ALB-Application-LB.452102c434d4093a\_20260916T1020Z\_52.5.185.34\_kik04igi.log.gz

Info

Copy S3 URI

Download

Open

Object actions

Properties

Permissions

Versions

Object overview

Owner

3c230a6da1d8b04e69c4d1dc392ca896c9876479133847483429b37a9182e61

AWS Region

US East (N. Virginia) us-east-1

Last modified

September 16, 2026, 15:50:11 (UTC+05:30)

Size

1.7 KB

Type

gz

Key

AWSLogs/33116047691/elasticloadbalancing/us-east-1/2026/09/16/33116047

S3 URI

s3://alb-access-logs-asif-2026/AWSLogs/33116047691/elasticloadbalancing/us-east-1/2026/09/16/33116047691\_elasticloadbalancing\_us-east-1\_app.ALB-Application-LB.452102c434d4093a\_20260916T1020Z\_52.5.185.34\_kik04igi.log.gz

Amazon Resource Name (ARN)

arn:aws:s3::alb-access-logs-asif-2026/AWSLogs/33116047691/elasticloadbalancing/us-east-1/2026/09/16/33116047691\_elasticloadbalancing\_us-east-1\_app.ALB-Application-LB.452102c434d4093a\_20260916T1020Z\_52.5.185.34\_kik04igi.log.gz

Entity tag (Etag)

2a43116dc8bb2c6ae533dd6e43a9a396

Object URL

https://alb-access-logs-asif-2026.s3.us-east-1.amazonaws.com/AWSLogs/33116047691/elasticloadbalancing/us-east-1/2026/09/16/33116047691\_elasticloadbalancing\_us-east-1\_app.ALB-Application-LB.452102c434d4093a\_20260916T1020Z\_52.5.185.34\_kik04igi.log.gz

## Result

The Application Load Balancer access logging was successfully configured with the S3 bucket alb-access-logs-asif-2026. AWS successfully created the ELBAccessLogTestFile, confirming that the ALB log delivery service has permission to write to the bucket.

## Conclusion

Successfully configured **Application Load Balancer access logs to be delivered to Amazon S3**. The S3 bucket provides centralized storage for ALB access logs, which can be used for traffic analysis, troubleshooting, auditing, and monitoring.